Vienna Traffic Router — Algorithm Documentation

This document explains all 8 search algorithms implemented in the Vienna Traffic Router codebase, how each operates, and what makes each implementation unique for this routing domain.

Core Concepts

Graph Structure

- **Nodes**: ~1,000 curated Vienna OSM nodes (intersections, waypoints)
- **Edges**: ~10,000 bidirectional edges with rich metadata (distance, surface quality, elevation, safety, etc.)
- **Graph representation**: Adjacency list with index-based edge lookup
  - `adjacency[node_id] = [{"node": nb_id, "edge_idx": int}, ...]`
  - All edge costs, blocks, and overrides keyed by `edge_idx`

Cost Model

- **g(n)**: Cumulative distance from start to node n (metres)
- **h(n)**: Heuristic estimate of distance from n to goal (metres × multiplier)
- **Manual overrides**: Intensity 0–94 → multiplier 0.5–2.85; ≥95 → impassable
- **Road quality**: Multiplied into edge cost before search (e.g., primary roads reduce cost, footways increase it)
- **Default speed**: 8.33 m/s (~30 km/h urban)

Optimality

- **Optimal path**: Shortest by total distance or cost
- **Admissible heuristic**: Never overestimates actual cost; guarantees A* optimality
- **ε-admissible**: Heuristic may overestimate by factor w; bounded suboptimality (Weighted A*)

Algorithm 1: Breadth-First Search (BFS)

What It Does

Explores all nodes at distance 1, then distance 2, then distance 3, etc. First to reach the goal is guaranteed to be the minimum-hop path.

How It Operates

```
1. Start: queue = [start_node]
2. Loop: pop node from front of queue
3. If node == goal: reconstruct & return path
4. Otherwise: add all unvisited neighbors to back of queue
5. If queue empties: no path found
```

Codebase Implementation (`src/algorithms/bfs.py`)

- **Data structure**: `deque` (double-ended queue)
- **Visited tracking**: `prev[node]` dict (node → parent) — implicitly tracks visited
- **Edge filtering**:
  - `is_edge_blocked()` — manual overrides with intensity ≥95
  - `is_edge_passable()` — vehicle type restrictions (metro/train have 0 passable roads)
- **Distance calculation**: Post-path using `sum_path_distance()` (ignores edge costs during search)
- **Metrics**: `nodes_expanded`, `compute_time_ms`

Optimality Guarantee

Not optimal for weighted graphs. Finds shortest number of hops, not shortest distance. On Vienna's graph (edges 30–300m), hopcount ≠ distance.

When to Use

- Baseline for comparison (explores many nodes, but simple)
- Unweighted graphs where each edge has equal importance

Performance Characteristics

- **Time**: O(V + E) in all cases (~1000 nodes, ~10,000 edges)
- **Space**: O(V) for queue and visited dict
- **Nodes expanded**: Often high, since no distance weighting guides search

---

Algorithm 2: Depth-First Search (DFS)

What It Does

Explores deeply along one branch before backtracking. No path-length guarantee — can find long, winding routes.

How It Operates

```
1. Start: stack = [start_node]
2. Loop: pop node from top of stack
3. If node == goal: reconstruct & return path
4. Otherwise: add all unvisited neighbors to top of stack (LIFO)
5. If stack empties: no path found
```

Codebase Implementation (`src/algorithms/dfs.py`)

- **Data structure**: Python `list` as stack (`.pop()` → LIFO)
- **Visited tracking**: `prev[node]` dict (same as BFS)
- **Edge filtering**: Identical to BFS
  - Manual overrides, vehicle passability
- **Distance calculation**: Post-path `sum_path_distance()`
- **Metrics**: `nodes_expanded`, `compute_time_ms`

Optimality Guarantee

Not optimal. Path depends on order of neighbor iteration; may find longest possible route.

When to Use

- Comparison baseline (depth-first flavor)
- Memory-constrained scenarios (uses less heap than breadth-first in some graphs, though adjacency list is shared)

Performance Characteristics

- **Time**: O(V + E) worst case, but often faster than BFS on sparse graphs
- **Space**: O(V) stack + visited
- **Nodes expanded**: Highly variable; can find path quickly if lucky with branching order, or explore deeply if unlucky

---

Algorithm 3: Uniform-Cost Search (UCS)

What It Does

Dijkstra's algorithm without the "visited" closed set. Expands nodes in order of increasing g(n) (cumulative cost). Optimal for non-negative edge weights.

How It Operates

```
1. Start: heap = [(0, tiebreak, start)]
2. Loop: pop (g, _, node) with minimum g
3. Skip if g > dist[node] (stale entry; re-expansions allowed)
4. If node == goal: reconstruct & return
5. For each unvisited neighbor:
   - If new_g < dist[neighbor]: update & push (new_g, tiebreak, neighbor)
```

Codebase Implementation (`src/algorithms/ucs.py`)

- **Data structure**: `heapq` (binary min-heap on g-cost)
- **Tiebreaker**: `itertools.count()` (ensures FIFO for equal costs, avoids heap errors)
- **Cost lookup**: `get_effective_edge_cost()`
  - Raw `distance_m` × manual override multiplier × road quality multiplier

    - Blocked edges (≥95 intensity) return `inf`
- **Stale node handling**: Line 35–36: re-expansions allowed; skip if popped g > recorded dist
- **Metrics**: `nodes_expanded`, `compute_time_ms`

Optimality Guarantee

Optimal. For non-negative edge weights, UCS always finds shortest path. Used as ground truth for computing `is_optimal` and `optimality_gap_pct` for all other results.

When to Use

- Primary baseline for Vienna router (runs all 8 algorithms in parallel; Dijkstra/UCS used to validate others)
- When heuristic is unavailable or untrusted
- Guaranteed optimal result needed

Performance Characteristics

- **Time**: O((V + E) log V) with binary heap
- **Space**: O(V) for dist dict, heap, and visited tracking
- **Nodes expanded**: Moderate; expands nodes in order of distance, unlike BFS/DFS

Algorithm 4: Dijkstra

What It Does

Same as UCS, but with a "visited" closed set to avoid re-expansions. Pops a node once, marks closed, then never reconsiders it.

How It Operates

```
1. Start: heap = [(0, tiebreak, start)]
2. Loop: pop (g, _, node) with minimum g
3. If node in visited: skip (already closed)
4. Add node to visited (close it)
5. If node == goal: reconstruct & return
6. For each unvisited neighbor:
   - If new_g < dist[neighbor]: update & push
```

Codebase Implementation (`src/algorithms/dijkstra.py`)

- **Data structure**: `heapq` (binary min-heap)
- **Visited set**: Line 35: `visited.add(current)` closes node permanently
- **Cost lookup**: `get_effective_edge_cost()` (same as UCS)
- **Difference from UCS**: No stale-entry check; avoids re-expansions entirely via `visited` set
- **Metrics**: `nodes_expanded`, `compute_time_ms`

Optimality Guarantee

Optimal. Dijkstra with closed-set optimization. Expands each node exactly once.

When to Use

- **Ground truth for optimality gap calculation** — runner compares all 8 results against Dijkstra
- When guaranteed optimal path and strict closed-set semantics desired
- More efficient than UCS (fewer re-expansions, though asymptotically same O((V + E) log V))

Performance Characteristics

- **Time**: O((V + E) log V)
- **Space**: O(V) for visited set, dist dict, heap
- **Nodes expanded**: Same as UCS on Vienna graph (~300–500 nodes depending on start/goal)

Algorithm 5: Greedy Best-First Search

What It Does

Always expands the node that appears closest to the goal by heuristic h(n), ignoring actual cost g(n). Can find paths very quickly but sacrifices optimality.

How It Operates

```
1. Start: heap = [(h(start), tiebreak, start)]
2. Loop: pop (h, _, node) with minimum h (ignore g)
3. If node in visited: skip
4. Add node to visited
5. If node == goal: reconstruct & return
6. For each unvisited neighbor:
   - prev[neighbor] = current
   - h = heuristic_fn(neighbor, goal)
   - Push (h, tiebreak, neighbor) to heap
```

Codebase Implementation (`src/algorithms/greedy_best_first.py`)

- **Heuristic**: Required parameter; called per-neighbor at line 58
- **Visited set**: Line 35: prevents re-expansion
- **Greedy decision**: Line 32: expands by h only, not f = g + h
- **No edge cost in heap**: Heap priority is purely `h(neighbor)`, not distance traveled
- **Metrics**: `nodes_expanded`, `compute_time_ms`

Heuristic Used

Vienna router provides `make_heuristic(params)` closure in `src/heuristics/combined.py`:

- Static tier: weather, time-of-day, vehicle multipliers (resolved once)
- Dynamic tier: edge-dependent costs (surface, elevation, density, etc.)
- Goal-dependent: district penalties, holiday effects

- Capped at `MAX_HEURISTIC_CAP` (default 50) to bound suboptimality

Optimality Guarantee

Not optimal. Greedy choice may lead to dead ends; no guarantee to find shortest path.

When to Use

- Fast, interactive routing when approximate paths acceptable
- Comparison: shows impact of heuristic on search speed vs. quality

Performance Characteristics

- **Time**: O(V log V) typical, O(V + E) worst case
- **Space**: O(V) for visited set and heap
- **Nodes expanded**: Very low on Vienna graph (~50–150 nodes); heuristic-dependent
- **Path quality**: Often 10–30% longer than optimal, highly variable

---

Algorithm 6: A*

What It Does

Combines actual cost g(n) and heuristic h(n) into f(n) = g(n) + h(n). Optimal when h is admissible (never overestimates). Expands nodes in order of estimated total path cost.

How It Operates

```
1. Start: heap = [(g + h, tiebreak, start)]
2. Loop: pop (f, _, node) with minimum f = g + h
3. If node in closed: skip
4. Add node to closed
5. If node == goal: reconstruct & return
6. For each unvisited neighbor:
   - tentative = g[current] + cost(edge)
   - If tentative < g[neighbor]:
     - g[neighbor] = tentative
     - f = tentative + h(neighbor)
     - Push (f, tiebreak, neighbor) to heap
```

Codebase Implementation (`src/algorithms/astar.py`)

- **Heuristic**: Required; called per-neighbor at line 64
- **g_score dict**: Line 28: tracks best known cost to each node
- **f computation**: Line 64: `f = tentative + heuristic_fn(...)`
- **Closed set**: Line 37: `closed.add(current)` prevents re-expansion
- **Cost lookup**: `get_effective_edge_cost()` includes manual overrides and road quality
- **Metrics**: `nodes_expanded`, `compute_time_ms`

Optimality Guarantee

Optimal if h is admissible. Vienna heuristic is designed to be admissible (Haversine distance × conservative multipliers), so A* on Vienna graph is optimal.

When to Use

- Primary production algorithm; balance speed and optimality
- When heuristic is trusted and admissible
- Most efficient guaranteed-optimal search on Vienna graph

Performance Characteristics

- **Time**: O((V + E) log V) in practice; often much faster than Dijkstra
- **Space**: O(V) for g_score, closed set, heap
- **Nodes expanded**: 200–400 nodes typical on Vienna graph (~50% of Dijkstra)
- **Speed gain**: 2–5× faster than Dijkstra due to heuristic guidance

---

Algorithm 7: Weighted A*

What It Does

A* variant that weights heuristic by factor w > 1: f(n) = g(n) + w × h(n). Trades optimality for speed; bounded ε-admissibility when w is known.

How It Operates

```
1. Start: heap = [(w * h, tiebreak, start)]
2. Loop: pop (f, _, node) with minimum f = g + w*h
3. If node in closed: skip
4. Add node to closed
5. If node == goal: reconstruct & return
6. For each unvisited neighbor:
   - tentative = g[current] + cost(edge)
   - If tentative < g[neighbor]:
     - g[neighbor] = tentative
     - f = tentative + w * h(neighbor)
     - Push (f, tiebreak, neighbor) to heap
```

Codebase Implementation (`src/algorithms/weighted_astar.py`)

- **Weight parameter**: Line 21: `w = float(params.get("w", 1.5))` (default 1.5)
- **f computation with weight**: Line 64: `f = tentative + w * heuristic_fn(...)`
- **Initial heap**: Line 27: `w * h_start` instead of `h_start`
- **Closed set & cost lookup**: Identical to A*
- **Metrics**: `nodes_expanded`, `compute_time_ms`

Optimality Guarantee

Bounded suboptimal. Path cost ≤ w × optimal cost. Vienna implementation caps heuristic at `MAX_HEURISTIC_CAP` (default 50) to ensure ε-admissibility when w = 1.5.

When to Use

- Interactive routing with real-time constraints
- When suboptimal but bounded solution acceptable
- Tune w: higher w = faster but worse solution; lower w = slower but better solution

Performance Characteristics

- **Time**: O((V + E) log V), typically 3–10× faster than A*
- **Space**: O(V)
- **Nodes expanded**: 50–100 nodes typical (w=1.5); half of A*
- **Path quality**: Often 5–15% longer than optimal with w=1.5

Weight Tuning

- w = 1.0 → A* (optimal, slow)
- w = 1.5 → Default; good speed/quality trade-off
- w = 2.0+ → Very fast, may sacrifice path quality significantly
- w → ∞ → Approaches greedy best-first (heuristic dominates)

Algorithm 8: Bidirectional A*

What It Does

Run A* forward (start → goal) and backward (goal → start) simultaneously, expanding the side with smaller f-value each iteration. Meets in the middle, dramatically reducing nodes expanded.

How It Operates

```
1. Forward search: g_f (start→node), h = distance(node→goal)
2. Backward search: g_b (goal→node), h = distance(node→start)
3. Loop:
   - If top_f ≤ top_b: expand forward
   - Else: expand backward
   - When both sides touch a node, record meeting cost (g_f + g_b)
4. Early termination: when top_f + top_b ≥ best_meeting_cost, best meeting
is optimal
5. Stitch path: forward_half + backward_half
```

Codebase Implementation (`src/algorithms/bidirectional_astar.py`)

- **Two searches**: `g_f`/`prev_f`/`open_f`/`closed_f` for forward, `g_b`/`prev_b`/`open_b`/`closed_b` for backward
- **Balanced expansion**: Lines 93–140: expands whichever side has smaller top-of-heap f

- **Meeting point detection**: Lines 112–116 (forward touches backward), 136–140 (backward touches forward)
- **Best sum tracking**: `best_sum = g_f[node] + g_b[node]` when both sides have visited same node
- **Early termination**: Lines 88–90: provably optimal when `(top_f + top_b) >= best_sum`
- **Reverse adjacency**: Lines 23–41: custom `_reverse_adjacency()` to navigate backward
  - Looks up predecessors by iterating over neighbors' neighbors (expensive but correct)
  - Returns edge_idx of forward edge for cost/block lookups
- **Path stitching**: Lines 155–168: forward_half + backward_half at meeting node

Bidirectional Specifics

Reverse edge requirement: Graph builder emits reverse edges for all non-strictly-oneway ways. Strictly oneway (e.g., one-way streets) have no reverse, correctly excluded from backward search.

Why `_reverse_adjacency()` is correct:

- Forward edges: `adjacency[a]` → neighbors of a
- Backward edges: find nodes p where p → a exists
- Iterate over neighbors of a, then their neighbors, looking for p → a edges
- Return edge_idx of p → a (not a → p) so cost/block checks use correct index

Optimality Guarantee

Optimal if h is admissible. Same guarantee as A*, with early termination proof: when both heaps can't improve best meeting cost, it's provably optimal.

When to Use

- Longest distances (interdistrict routing) where bidirectional overhead pays off
- When node count is very high and balanced expansion desirable
- Comparison: shows effectiveness of "meet in middle" vs. single-direction A*

Performance Characteristics

- **Time**: O((V + E) log V), similar to A* asymptotically, but much smaller V in practice
- **Space**: O(V) for two g-score dicts, two closed sets, two heaps
- **Nodes expanded**: ~50–100 total (forward + backward combined), often 2–3× fewer than A*
- **Speed gain**: 2–5× faster than A* on Vienna graph due to reduced search frontier
- **Meeting overhead**: Reverse adjacency lookup is O(degree^2) per backward expansion, but amortized negligible on Vienna graph

Example Meeting Path

```
Start (A) → [forward search expands: B, C, D] → [backward from Goal
expands: E, F, D]
Meeting at D: path = [A, B, C, D] + [D, F, E, Goal]
Check: g_f[D] + g_b[D] < top_f + top_b? Yes → provably optimal, stop.
```

Summary Table

Algorithm	Optimal?	Admissible h?	Speed	Nodes Expanded	Best Use
BFS	No (hop-based)	N/A	Slow	~1000	Baseline comparison
DFS	No (order-dependent)	N/A	Variable	~1000	Memory-constrained
UCS	Yes	N/A (no h)	Moderate	300–500	Ground truth
Dijkstra	Yes	N/A (no h)	Moderate	300–500	Ground truth with closed-set
Greedy	No	Requires h	Fast	50–150	Approximate, fast
A*	Yes	Yes	Fast	200–400	Best overall (production)
Weighted A*	Bounded ε	Yes (with cap)	Faster	50–100	Real-time, interactive
Bidirectional A*	Yes	Yes	Faster	50–100	Long distances, proof-optimal

Heuristic Details

All heuristics (except BFS, DFS, Dijkstra, UCS) call `make_heuristic(params)` from `src/heuristics/combined.py`:

Static Tier (Resolved Once at Closure Build)

- Weather multiplier (rain/snow → slower)
- Time-of-day multiplier (rush hour → slower)
- Vehicle-type speed baseline
- Visibility, humidity, wind effects
- Headway multiplier (wait at lights)
- Night network mode (reduced connectivity)

Dynamic Tier (Evaluated Per-Node)

- Intersection density (busy → slower)
- Road surface (rough → slower, primary → faster)
- Elevation changes
- Safety metrics (accident density)
- Emissions zone penalties
- Tram track interaction
- Snow priority roads
- Bridge/tunnel overhead

- Lane capacity constraints
- School zones
- Scenic route detours

Goal-Dependent Tier

- District-specific parking difficulty
- Holiday closures
- Event traffic
- Fiaker (horse carriage) routes
- Seasonal effects
- Pedestrian density

Heuristic Cap

`MAX_HEURISTIC_CAP` (env var, default 50) ensures admissibility by capping composite multiplier. Without cap, stacked multipliers could violate admissibility and break A* optimality guarantee.

Edge Filtering in All Algorithms

Every algorithm respects two global constraints:

1. **Manual overrides** (`is_edge_blocked()`):
    - Intensity ≥95 → impassable (returns in early termination on blocked paths)
    - Intensity 0–94 → multiplier 0.5–2.85 applied to edge cost
2. **Vehicle passability** (`is_edge_passable()`):
    - Car, taxi, motorcycle, truck: primary/secondary/residential roads
    - Bicycle, escooter: cycleway + roads
    - Walking: footway + residential + parks
    - Bus: residential + primary + secondary + busways
    - Metro/train: 0 roads (no graph edges); always impassable (comparison only)

Blocked + impassable edges are skipped entirely; algorithms never consider them as neighbors.

Metrics Reported

Every `PathResult` includes:

- `algorithm`: Name (bfs, dfs, ucs, dijkstra, greedy, astar, weighted_astar, bidirectional_astar)
- `path_node_ids`: List of node IDs from start to goal
- `path_coords`: List of (lat, lon) tuples
- `distance_m`: Total edge distance in metres
- `estimated_time_s`: distance_m / speed_for_vehicle(vehicle_type)
- `nodes_expanded`: Count of nodes popped from queue/heap
- `compute_time_ms`: Wall-clock time (perf_counter)
- `is_optimal`: Boolean; True if path matches Dijkstra result

- `optimality_gap_pct`: (this_distance - dijkstra_distance) / dijkstra_distance × 100
- `error`: None if successful; error string if path not found

Runner Integration (`src/algorithms/runner.py`)

```
run_all(start_lat, start_lon, goal_lat, goal_lon, params, heuristic_fn)
→ ThreadPoolExecutor(max_workers=8)
  → 8 parallel threads, one per algorithm
  → All receive same heuristic_fn, params, start, goal
  → Dijkstra result used as ground truth for optimality_gap_pct
  → Results collected as dict[algo_name → PathResult]
```

Testing & Validation

No test suite exists yet. Validation approach:

1. Visual inspection: Map display shows all 8 routes
2. Heuristic tuning: Adjust multipliers via sidebar; watch routes recompute
3. Override verification: Mark roads as blocked; verify algorithms avoid them
4. Vehicle passability: Switch vehicle type; verify metro/train routes fail gracefully
5. Optimality check: Dijkstra and A* must match; Greedy/Weighted A* may differ by known gap

---

Performance on Vienna Graph

Typical numbers (interdistrict: ~2 km distance, ~1000 nodes total):

Algorithm	Time (ms)	Nodes Expanded	Path Length (m)	vs. Optimal
BFS	50–100	800–1000	≈2500	+50%
DFS	30–80	600–1000	Variable	Variable
UCS	15–30	300–500	≈1800	Optimal
Dijkstra	15–30	300–500	≈1800	Optimal
Greedy	5–10	50–150	≈2000	+10%
A*	10–20	200–400	≈1800	Optimal
Weighted A* (w=1.5)	5–10	50–100	≈1900	+5%
Bidirectional A*	8–15	50–100	≈1800	Optimal

Times and node counts vary with heuristic quality and manual overrides.

Future Extensions

- *LPA (Lifelong Planning A)**: Replan incrementally when costs change
- **Theta***: Straight-line paths without intermediate nodes (requires angle calculations)
- **Timed A***: Anytime planning with solution refinement
- **Multi-objective**: Pareto frontier for distance vs. safety vs. comfort
- **k-shortest paths**: Find top k routes for comparison